
Learning What to Learn: Hippocampal Memory Consolidation for Continual Model Adaptation

Anonymous Author(s)

Affiliation

Address

email

Abstract

Contemporary large language models are trained on internet-scale corpora and scaled to massive parameter regimes, incurring substantial costs in training, deployment, and continual adaptation. Human cognition, by contrast, is selective and experience-driven, relying on hippocampal mechanisms that preferentially encode salient, novel, and behaviorally relevant experiences and coordinate their gradual consolidation into long-term neocortical memory. We propose **Hippocampus-Augmented Transformer (HAT)**, a memory-centric continual learning mechanism inspired by hippocampal-neocortical consolidation in the mammalian brain. HAT enables language models to learn *what to learn* from interaction by coupling online experience acquisition with offline memory consolidation. During interaction, a base model (the *Cortex*) processes inputs while a *Hippocampus Agent* encodes interactions and feedback into structured memory traces and selectively retains informative experiences based on uncertainty, novelty, and supervision signals, optionally querying an external oracle when needed. Selected experiences are progressively consolidated into a persistent *Neocortex* memory and subsequently replayed during slow-wave sleep (SWS), where the Cortex is updated through parameter-efficient fine-tuning. This wake-sleep separation transforms sparse, interaction-driven feedback into reusable training signals, enabling continual self-improvement without indiscriminate data accumulation. We evaluate HAT on knowledge-intensive question answering, instruction following, and personalization benchmarks, demonstrating consistent improvements over fine-tuning and retrieval-augmented baselines under continual adaptation settings.

1 Introduction

Large language models (LLMs) have demonstrated remarkable capabilities across a broad spectrum of natural language tasks [Brown et al., 2020, Touvron et al., 2023, OpenAI, 2023]. Yet a fundamental tension remains: once training concludes, a model’s knowledge is *frozen*, and adapting it to new domains, user preferences, or corrected information requires costly retraining on carefully curated data. This limitation is particularly acute for lightweight, on-device language models in the 0.5–10 B parameter range, where users interact with a personal assistant that must evolve over time but lacks the infrastructure for large-scale retraining.

Existing approaches to post-deployment adaptation fall broadly into two categories. *Retrieval-augmented generation* (RAG) methods [Lewis et al., 2020, Guu et al., 2020] augment the model’s context with external knowledge at inference time, but they do not update the model’s parameters and thus cannot internalise corrections or new skills. *Continual fine-tuning* methods [Kirkpatrick et al., 2017, De Lange et al., 2021] update model weights on new data, but they typically require hand-curated datasets and are susceptible to catastrophic forgetting when data is noisy or unbalanced. Neither paradigm addresses the upstream question: *what should a model learn from?*

In biological intelligence, this problem is solved elegantly. The mammalian hippocampus does not store all sensory experience; instead, it selectively encodes salient events and, during sleep, replays and consolidates them into long-term neocortical memory [McClelland et al., 1995, Kumaran et al., 2016]. This process of *selective memory consolidation* ensures that only high-value experiences—those that are novel, surprising, or corrective—are used to update long-term knowledge.

Inspired by this principle, we propose **HAT (Hippocampus-Augmented Transformer)**, a multi-agent framework that equips a lightweight language model with the ability to *learn what to learn* from its own interactions. HAT introduces a dedicated *Hippocampus Agent* that monitors the base model’s (the *Cortex*) interactions with users, abstracts raw dialogues into structured memory traces, selects high-value experiences using uncertainty, feedback, and novelty signals, and constructs training-ready replay data. During offline phases—analogue to *slow-wave sleep* (SWS) in the brain—curated experiences are replayed to fine-tune the Cortex, producing an improved model that re-enters the interaction loop. When the Cortex is uncertain or produces errors, HAT can also consult an external *Oracle* (a stronger teacher model), enabling a form of on-demand knowledge distillation.

Contributions. Our contributions are three-fold:

1. We formalise the problem of *selective experience curation* for self-improving language models and propose HAT, a memory-centric framework that decouples experience acquisition from parameter optimisation.
2. We design a Hippocampus Agent with three composable stages—*abstraction*, *selection*, and *replay construction*—that convert raw interactions into high-quality training signals using uncertainty estimation, user feedback, and novelty detection.
3. We conduct extensive experiments on knowledge-intensive QA, instruction following, and personalisation benchmarks, demonstrating that HAT achieves consistent improvements in sample efficiency, error-rate reduction, and long-horizon adaptation over strong fine-tuning and retrieval-augmented baselines.

2 Related Work

2.1 Continual Learning for Language Models

Continual learning (CL) aims to update model parameters on a stream of tasks without catastrophic forgetting [Kirkpatrick et al., 2017, Zenke et al., 2017]. Regularisation-based methods penalise changes to important weights [Kirkpatrick et al., 2017, Li and Hoiem, 2017], while replay-based methods store or generate past examples for interleaved training [De Lange et al., 2021, Scialom et al., 2022]. Recent work has extended CL to large language models, studying instruction tuning across sequential tasks [Wu et al., 2024] and domain-incremental adaptation [Ke et al., 2023]. However, existing CL methods assume a *given* data stream and do not address the problem of *which* experiences should be retained. HAT complements CL by introducing a principled experience selection mechanism upstream of parameter updates.

2.2 Memory-Augmented Neural Networks

Memory-augmented architectures equip neural networks with external memory stores that can be read from and written to [Graves et al., 2014, Sukhbaatar et al., 2015]. More recent work combines retrieval mechanisms with language models [Borgeaud et al., 2022, Zhong et al., 2022], enabling access to a large knowledge base at inference time. While these approaches augment *inference*, they do not leverage memory for *learning*: retrieved information is used contextually but does not update the model’s parameters. In contrast, HAT uses its memory store (the Neocortex) as a curated training set that feeds back into parameter optimisation.

2.3 Retrieval-Augmented Generation

Retrieval-augmented generation (RAG) methods prepend retrieved passages to the model’s input context [Lewis et al., 2020, Guu et al., 2020, Izacard et al., 2023]. RAG is effective for knowledge-intensive tasks but inherits the limitations of fixed-context approaches: the model cannot *learn* from

retrieved content, context windows are bounded, and retrieval quality degrades when the knowledge base is noisy. HAT can be seen as going beyond RAG by closing the loop from retrieval to training: high-value interactions—including oracle-provided corrections—are not only used at inference time but are consolidated into the model’s parameters.

2.4 Knowledge Distillation and Self-Improvement

Knowledge distillation transfers knowledge from a teacher model to a student [Hinton et al., 2015, Sanh et al., 2019]. Recent work explores *self-improving* language models that generate their own training data [Huang et al., 2023, Yuan et al., 2024, Chen et al., 2024] or use reinforcement learning from human/AI feedback [Ouyang et al., 2022, Bai et al., 2022]. These methods typically apply distillation or self-play uniformly, without selecting *which* experiences are most informative. HAT’s Oracle consultation mechanism is a form of selective, on-demand distillation: the teacher is queried only when the student model is uncertain or has been corrected by the user, ensuring efficient use of the oracle’s capacity.

2.5 Complementary Learning Systems Theory

Complementary learning systems (CLS) theory posits that the brain uses two interacting memory systems: the hippocampus for rapid encoding of specific episodes and the neocortex for slow extraction of statistical regularities [McClelland et al., 1995, Kumaran et al., 2016]. Sleep replay, particularly during slow-wave sleep, is hypothesised to mediate the transfer from hippocampus to neocortex [Rasch and Born, 2013, Walker, 2017]. Several works in machine learning have drawn on CLS for continual learning [Shin et al., 2017, van de Ven et al., 2020, Arani et al., 2022], but they focus on image classification and do not address the unique challenges of language model self-improvement from interactive experience. HAT directly operationalises the CLS framework for LLMs, with an explicit hippocampal selection stage, a neocortical memory store, and an offline SWS training phase.

3 HAT: Hippocampus-Augmented Transformer

We introduce **Hippocampus-Augmented Transformer (HAT)**, a memory-centric continual learning architecture for language models that separates *interaction*, *memory consolidation*, and *model adaptation*. The motivating observation is that most online interactions are not equally useful for future learning: many are routine, redundant, or already within the model’s competence, whereas a small subset exposes epistemic gaps, user-specific corrections, or reusable knowledge. Rather than updating model parameters after every turn, HAT first decides whether an interaction should become a persistent training signal and only later replays accepted memories during an offline slow-wave-sleep (SWS) phase.

The design follows a hippocampal–neocortical division of labor. The **Cortex** $\mathcal{M}_\theta$ is the online language model that produces responses and exposes confidence information. The **Hippocampus Agent** $\mathcal{H}$ converts selected interactions into compact, canonical memory traces and decides how each trace should modify long-term memory. The **Neocortex** $\mathcal{N}$ is an appendable and revisable store of curated traces, indexed by semantic embeddings for nearest-neighbor routing. The **SWS Trainer** $\mathcal{SWS}$ samples high-value traces from the Neocortex and converts them into supervised replay batches for parameter-efficient adaptation. Optionally, an external **Oracle** $\mathcal{O}$ is queried only on high-uncertainty turns, providing budgeted supervision at the frontier of the model’s competence.

3.1 Selective Continual Consolidation

Let the wake stream at cycle t be

$$\mathcal{D}^{(t)} = \{x_i^{(t)}\}_{i=1}^{T_t}, \quad x_i = (c_i, q_i, y_i), \quad (1)$$

where c_i is optional conversational context, q_i is the user query, and y_i is the Cortex response. User corrections are not assumed to arrive as a clean scalar label; in realistic chat settings they may appear as subsequent turns in the same session and must be interpreted in context. HAT therefore maintains two distinct data regimes: raw session history, which preserves what happened, and curated Neocortex memory, which contains only authorized training traces.

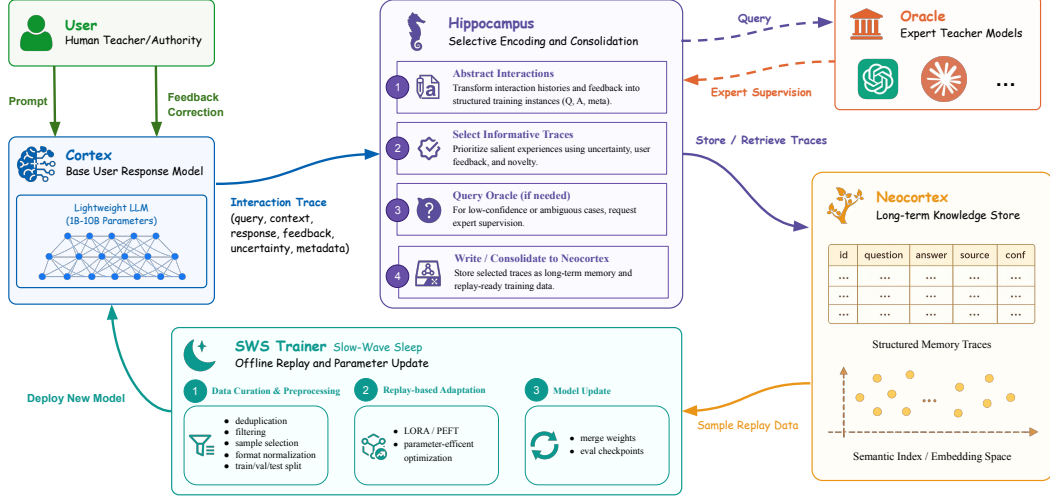


Figure 1: Overview of HAT. During wake-phase interaction, the Cortex answers a user query and the Hippocampus estimates uncertainty, triages the turn, extracts one or more canonical knowledge traces, and routes each trace to either CREATE a new memory or REVISE an existing one using a vector-indexed Neocortex. Accepted memories are stored as replay-ready supervised examples and later sampled during slow-wave sleep, where the SWS Trainer updates the Cortex without requiring online parameter updates after every interaction.

134 The ideal memory update would retain the subset of experience that maximizes downstream utility
 135 under a bounded replay budget:

$$\mathcal{N}^{*(t)} = \arg \max_{\mathcal{N}' \in \mathcal{U}(\mathcal{N}^{(t-1)}, \mathcal{D}^{(t)})} \mathbb{E}_{(q,a) \sim p_{\text{replay}}(\cdot | \mathcal{N}')} [\Delta \mathcal{L}_{\text{future}}(\theta; q, a)] \quad \text{s.t.} \quad |\mathcal{N}'| \leq B, \quad (2)$$

136 where $\mathcal{U}$ is the set of admissible CREATE and REVISE operations. The expected future loss reduction
 137 is unobservable at write time, so HAT replaces it with a structured surrogate:

$$\underbrace{U(x) \geq \tau_w}_{\text{eligibility}} \implies \underbrace{\mathcal{H}_{\text{abs}}(x)}_{\text{canonical traces}} \implies \underbrace{R(m, \mathcal{N}) \in \{\text{CREATE}, \text{REVISE}\}}_{\text{semantic routing}} \implies \underbrace{\text{Write}(m, d)}_{\text{authorized memory transition}}. \quad (3)$$

138 This factorization is the core methodological choice. Uncertainty controls whether an interaction is
 139 worth spending consolidation compute on; abstraction produces canonical knowledge-level traces;
 140 geometric routing reconciles a trace with prior memory; and the write policy emits an explicit
 141 authority token required by the memory store.

142 3.2 Wake-Sleep Decomposition

143 At each cycle, HAT alternates between a wake operator $\mathcal{W}$ and a sleep operator $\mathcal{S}$:

$$\mathcal{N}^{(t)} = \mathcal{W}(\mathcal{N}^{(t-1)}, \mathcal{D}^{(t)}; \mathcal{M}_\theta, \mathcal{H}, \mathcal{O}), \quad (4)$$

$$\theta^{(t+1)} = \mathcal{S}(\theta^{(t)}, \mathcal{N}^{(t)}). \quad (5)$$

144 The wake operator does not update model parameters. It produces a sequence of accepted memory
 145 creations and revisions. The sleep operator samples from the resulting memory and performs batched
 146 empirical risk minimization. This decomposition avoids unstable online learning and makes the
 147 provenance of every replay example explicit.

148 3.3 Cortex: Online Model and Uncertainty Gate

149 The Cortex is an autoregressive language model $\mathcal{M}_\theta$ with parameters θ . Given context c and query q ,
150 it generates

$$p_\theta(y \mid c, q) = \prod_{j=1}^{|y|} p_\theta(y_j \mid c, q, y_{<j}). \quad (6)$$

151 In addition to generation, the Cortex provides an uncertainty score for the realized response. When
152 token log-probabilities are available, HAT uses one minus the geometric mean token probability:

$$U(x) = 1 - \exp\left(\frac{1}{|y|} \sum_{j=1}^{|y|} \log p_\theta(y_j \mid c, q, y_{<j})\right), \quad U(x) \in [0, 1]. \quad (7)$$

153 High-probability responses yield $U(x) \approx 0$, whereas low-confidence responses yield larger uncer-
154 tainty. For backends that do not expose token log-probabilities, the implementation falls back to a
155 calibrated constant, preserving the same interface while avoiding backend-specific failure modes.

156 Uncertainty is used as a computational gate:

$$g(x) = \mathbb{1}U(x) \geq \tau_w. \quad (8)$$

157 Only turns with $g(x) = 1$ are passed to the abstraction pipeline. Confident routine interactions remain
158 in raw session history but are not summarized, embedded, or written into curated memory.

159 3.4 Hippocampus Agent: Abstraction, Routing, and Authorization

160 The Hippocampus Agent implements the wake-time transition from raw interaction to curated
161 memory. Its current instantiation consists of optional oracle correction, LLM-based trace abstraction,
162 deterministic embedding routing, and an authorized write policy.

163 3.4.1 Budgeted Oracle Consultation

164 When an Oracle is configured, HAT consults it only for high-uncertainty interactions:

$$y^\mathcal{O} = \mathcal{O}(c, q) \quad \text{if} \quad U(x) > \tau_\mathcal{O}. \quad (9)$$

165 If the Oracle returns a nonempty correction, the target response used for downstream abstraction
166 is replaced by $y^\mathcal{O}$. Thus external supervision is acquired selectively rather than distilled over all
167 interactions. This mechanism allocates teacher calls to examples on which the student is least reliable.

168 3.4.2 Trace Abstraction

169 For every eligible interaction, the abstractor maps a verbose turn into zero or more canonical memory
170 traces:

$$\{m_{i,k}\}_{k=1}^{K_i} = \mathcal{H}_{\text{abs}}(c_i, q_i, y_i), \quad (10)$$

171 where $K_i = 0$ means that the turn is dropped. Each trace has the structured form

$$m = (q^*, a_{\text{cortex}}, a^*, r, \xi), \quad (11)$$

172 where q^* is a canonical query, a_{cortex} is the original model response, a^* is the replay target, r is an
173 optional rationale, and ξ stores provenance such as session id, interaction ids, uncertainty, oracle
174 usage, and routing metadata.

175 The production abstractor is a two-stage LLM judge. First, a triage prompt predicts whether the turn
176 contains a knowledge point worth remembering:

$$b_i = T_\phi(c_i, q_i, y_i) \in \{0, 1\}. \quad (12)$$

177 If $b_i = 0$, no trace is emitted. If $b_i = 1$, an extraction prompt emits a JSON-structured set of
178 independent knowledge points:

$$\{(q_{i,k}^*, a_{i,k}^*, r_{i,k})\}_{k=1}^{K_i} = E_\phi(c_i, q_i, y_i). \quad (13)$$

179 This design permits a single user turn to yield multiple replay examples while filtering small talk,
180 meta-comments, and uninformative exchanges. Crucially, the abstractor does not decide whether
181 a trace is new or should overwrite an existing memory; that decision is made by a deterministic
182 embedding router.

183 3.4.3 Geometric CREATE/REVISE Routing

184 Let $e(q) \in \mathbb{R}^d$ denote an L2-normalized embedding of a canonical query, and let the Neocortex
185 maintain a vector index

$$\mathcal{I}^{(t)} = \{(\text{id}_j, v_j) : v_j = e(q_j^*), \|v_j\|_2 = 1\}. \quad (14)$$

186 For a candidate trace m with query q^* , HAT retrieves its nearest existing memory:

$$j^* = \arg \max_{j \in \mathcal{I}^{(t)}} e(q^*)^\top v_j, \quad \rho(m) = e(q^*)^\top v_{j^*}. \quad (15)$$

187 Since all vectors are unit normalized, $\rho(m)$ is cosine similarity. The routing rule is

$$R(m) = \begin{cases} \text{REVISE}(\text{id}_{j^*}), & \rho(m) \geq \tau_r, \\ \text{CREATE}, & \rho(m) < \tau_r \text{ or } \mathcal{I}^{(t)} = \emptyset. \end{cases} \quad (16)$$

188 Equivalently, $1 - \rho(m)$ is a semantic novelty measure, but HAT uses it for routing rather than as a
189 linear term in the write score. This separation is important: uncertainty answers whether the current
190 turn is worth remembering, while geometric similarity answers whether it should become a new
191 memory or revise an existing one.

192 On the REVISE path, the matched trace is updated in place with the new canonical query, target
193 response, rationale, and interaction provenance; the previous target is retained in trace history. The
194 vector index entry for the matched trace is updated with the new query embedding. On the CREATE
195 path, a new trace id is appended to the Neocortex and its embedding is appended to the vector index.

196 3.4.4 Write Policy and Authorized Memory Transition

197 The write policy maps trace-level signals to a scalar decision. In the current implementation the score
198 is the uncertainty of the originating interaction:

$$s(m) = U(x(m)), \quad d(m) = (m.\text{id}, s(m), \tau_w, \mathbb{1}s(m) \geq \tau_w). \quad (17)$$

199 The decision $d(m)$ is an authorization token. The Neocortex accepts a write only if the decision is
200 positive and its trace id matches the candidate trace:

$$\text{Write}(m, d) \text{ is valid} \iff d.\text{accepted} = 1 \wedge d.\text{trace_id} = m.\text{id}. \quad (18)$$

201 This explicit write-token contract enforces the boundary between raw chat logs and curated training
202 data. It prevents accidental persistence of unapproved interactions and makes the raw-to-curated
203 transition auditable.

204 3.5 Neocortex: Curated Long-Term Memory

205 The Neocortex stores accepted traces as supervised fine-tuning rows plus sidecar metadata. For each
206 accepted trace m_i , the primary training record is

$$z_i = ([(\text{user}, q_i^*), (\text{assistant}, a_i^*)], \text{id}_i, s_i, \xi_i), \quad (19)$$

207 which is directly compatible with standard chat supervised fine-tuning pipelines. The sidecar trace
208 preserves richer information such as rationale, interaction history, oracle provenance, and routing
209 similarity.

210 The memory update induced by a candidate trace can be written abstractly as

$$\mathcal{N} \leftarrow \begin{cases} \mathcal{N} \cup \{m\}, & R(m) = \text{CREATE}, d(m).\text{accepted} = 1, \\ \text{Revise}(\mathcal{N}, j^*, m), & R(m) = \text{REVISE}, d(m).\text{accepted} = 1, \\ \mathcal{N}, & d(m).\text{accepted} = 0. \end{cases} \quad (20)$$

211 For replay, the Neocortex exposes priority sampling by write score:

$$\text{Sample}_k(\mathcal{N}) = \text{TopK}_{m \in \mathcal{N}} s(m), \quad (21)$$

212 with manual edit and delete operations treated as memory curation rather than new learning events.

3.6 Replay Construction and Slow-Wave Sleep Training

The Replay Builder converts each retained memory trace into one or more supervised examples. The default builder emits a single instruction–response pair:

$$\mathcal{R}(m) = \{(q^*, a^*, w, \text{id}_m, o_m)\}, \quad (22)$$

where w is an example weight and o_m indicates whether the target was Oracle-derived. At sleep cycle t , HAT samples traces from the Neocortex and constructs

$$\mathcal{B}^{(t)} = \bigcup_{m \in \text{Sample}_k(\mathcal{N}^{(t)})} \mathcal{R}(m). \quad (23)$$

The supervised SWS objective is the token-level negative log-likelihood over the replay batch:

$$\mathcal{L}_{\text{sup}}(\theta; \mathcal{B}) = -\frac{1}{|\mathcal{B}|} \sum_{(q,a) \in \mathcal{B}} \sum_{j=1}^{|a|} \log p_{\theta}(a_j \mid q, a_{<j}). \quad (24)$$

When Oracle-derived targets or stability constraints are available, the trainer may optimize the more general objective

$$\mathcal{L}_{\text{SWS}} = \mathcal{L}_{\text{sup}} + \lambda_{\text{KD}} \mathcal{L}_{\text{KD}} + \lambda_{\text{stab}} \mathcal{L}_{\text{stab}}, \quad (25)$$

where $\mathcal{L}_{\text{KD}}$ distills Oracle supervision and $\mathcal{L}_{\text{stab}}$ penalizes drift from previously important parameters. The parameter update is

$$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla_{\theta} \mathcal{L}_{\text{SWS}}(\theta^{(t)}; \mathcal{B}^{(t)}). \quad (26)$$

In deployment, this update can be instantiated with LoRA or adapter-based fine-tuning, keeping most Cortex parameters frozen and making SWS feasible for resource-constrained models.

3.7 Algorithm

Algorithm 1 summarizes the complete wake–sleep procedure. The key design choice is that abstraction, routing, authorization, and replay are separate operators. This modularity localizes failure modes: uncertain but uninformative turns are dropped by triage, duplicate knowledge revises existing traces rather than inflating memory, and only accepted write decisions can cross the raw-to-curated boundary.

3.8 Discussion: Why the Separation Matters

HAT deliberately avoids conflating three questions that are often merged in continual-learning systems. First, *eligibility*: uncertainty determines whether the interaction is worth spending consolidation compute on. Second, *semantic identity*: embedding similarity determines whether a trace expands memory coverage or revises an existing concept. Third, *parameter adaptation*: SWS replay determines how accepted memories affect the Cortex. This separation yields a practical continual learner whose memory is sparse, auditable, and replay-ready, while leaving the online model stable during user interaction.

4 Experiments

4.1 Benchmarks and Tasks

We evaluate HAT on three families of tasks that probe different aspects of self-improvement:

Knowledge-intensive question answering. We use Natural Questions (NQ) [Kwiatkowski et al., 2019] and TriviaQA [Joshi et al., 2017] in a *streaming* setup: questions arrive sequentially and the model must answer without access to a retrieval corpus. After each answer, the model may receive a binary correctness signal or an explicit user correction.

Instruction following. We adopt a subset of the Alpaca-Eval benchmark [Li et al., 2023] and MT-Bench [Zheng et al., 2023], where instructions are presented in temporal batches that gradually increase in complexity. This tests HAT’s ability to internalise and generalise from corrective feedback on instruction-following quality.

Algorithm 1: HATwake-sleep learning loop

Input: Initial Cortex $\mathcal{M}_\theta^{(0)}$, Neocortex $\mathcal{N}^{(0)}$, thresholds $\tau_w, \tau_\mathcal{O}, \tau_r$, optional Oracle $\mathcal{O}$

```
1 for cycle  $t = 0, 1, 2, \dots$  do
2   Collect wake interactions  $\mathcal{D}^{(t)}$ 
3   for each interaction  $x = (c, q, y) \in \mathcal{D}^{(t)}$  do
4     Estimate uncertainty  $U(x)$  using Eq. 7
5     if  $U(x) < \tau_w$  then
6       Skip abstraction and do not write memory
7     else
8       if Oracle is enabled and  $U(x) > \tau_\mathcal{O}$  then
9         Query  $\mathcal{O}(c, q)$  and replace the replay target if a correction is returned
10      Triage the turn; if it is knowledge-bearing, extract traces  $\{m_k\}_{k=1}^K$  as canonical
        ( $q^*, a^*$ ) pairs
11      for each trace  $m_k$  do
12        Compute write decision  $d(m_k)$  from Eq. 17
13        Embed  $q_k^*$  and retrieve nearest memory by Eq. 15
14        Route by Eq. 16: CREATE a new trace or REVISE the matched trace
15        If  $d(m_k)$  is accepted, apply the authorized Neocortex transition in Eq. 20
16  Sample replay batch  $\mathcal{B}^{(t)}$  using Eq. 21–23
17  Update Cortex parameters with the SWS objective in Eq. 25
```

250 **Personalisation.** We construct a personalisation benchmark based on LaMP [Salemi et al., 2024],
251 where a simulated user provides preferences and corrections over successive interactions. The model
252 is evaluated on how well it adapts to individual user style and factual preferences over time.

253 4.2 Baselines

254 We compare HAT against the following approaches:

- 255 • **Frozen:** the base Cortex model without any adaptation.
- 256 • **RAG** [Lewis et al., 2020]: retrieval-augmented generation using a dense retriever over the interac-
257 tion history.
- 258 • **Naïve Replay:** continual fine-tuning on all interaction logs without selection.
- 259 • **EWC Replay** [Kirkpatrick et al., 2017]: replay with elastic weight consolidation but without
260 experience curation.
- 261 • **DER++** [Buzzega et al., 2020]: dark experience replay with knowledge distillation, a strong
262 continual learning baseline.
- 263 • **Self-Play** [Chen et al., 2024]: the model generates and filters its own training data via self-
264 consistency.
- 265 • **Full Distillation:** fine-tuning the student on the oracle’s responses for *all* queries (upper bound on
266 oracle usage).

267 4.3 Implementation Details

268 **Base models.** We instantiate the Cortex with three model families at different scales: Qwen-2.5-
269 1.5B, Phi-3-mini-3.8B, and LLaMA-3-8B. All models are instruction-tuned variants.

270 **Oracle.** We use GPT-4o as the default oracle $\mathcal{O}$. We also report results with a smaller oracle
271 (LLaMA-3-70B) to study oracle quality sensitivity.

272 **Hippocampus Agent.** The abstraction module uses the Cortex itself with dedicated triage and
273 extraction prompts, making HAT lightweight and self-contained. Write eligibility is determined by

Table 1: Main results after 10 SWS cycles. We report accuracy (%) on knowledge QA (NQ / TriviaQA, exact match), instruction following (MT-Bench, GPT-4 judge score $\times 10$), and personalisation (LaMP, ROUGE-L $\times 100$). Best in **bold**; second best underlined. [†]Full Distillation uses the oracle for *every* query (upper bound).

Method	Knowledge QA			Instruction Following			Personalisation		
	1.5B	3.8B	8B	1.5B	3.8B	8B	1.5B	3.8B	8B
Frozen	–	–	–	–	–	–	–	–	–
RAG	–	–	–	–	–	–	–	–	–
Naïve Replay	–	–	–	–	–	–	–	–	–
EWC Replay	–	–	–	–	–	–	–	–	–
DER++	–	–	–	–	–	–	–	–	–
Self-Play	–	–	–	–	–	–	–	–	–
Full Distill. [†]	–	–	–	–	–	–	–	–	–
HAT (ours)	–	–	–	–	–	–	–	–	–

the log-probability uncertainty gate in Eq. (8). Semantic redundancy is handled separately by the embedding-based CREATE/REVISE router in Eq. (16), rather than being mixed into the write score.

Hyperparameters. We set the write threshold to $\tau_w = 0.3$ in Eq. (17) and the memory-routing threshold to $\tau_r = 0.85$ in Eq. (16), based on validation-set tuning. Oracle consultation is triggered when $U > \tau_O = 0.7$. SWS training uses LoRA (rank 16) with AdamW ($\eta = 2 \times 10^{-5}$), batch size 32, and 3 epochs per cycle. The EWC coefficient is $\mu = 0.1$. A SWS cycle is triggered every 500 interactions.

Evaluation protocol. We report metrics averaged over 3 random seeds. Each experiment simulates 5,000 sequential interactions (10 SWS cycles). We measure: (1) **accuracy** (exact match or ROUGE-L depending on task), (2) **error rate** (fraction of incorrect responses), (3) **sample efficiency** (accuracy gain per training example), and (4) **forgetting rate** (performance drop on earlier tasks after adaptation).

5 Results

5.1 Main Results

Table 1 summarises the performance of HAT and all baselines after 5,000 interactions (10 SWS cycles) across three benchmark families and three base model scales.

HAT consistently outperforms non-parametric baselines.

HAT surpasses continual learning baselines.

Selective curation closes the gap to full distillation.

5.2 Sample Efficiency

Figure 2 plots accuracy as a function of the number of training examples consumed.

5.3 Oracle Budget Analysis

A practical concern is the cost of oracle consultations. Table 2 reports performance and the number of oracle calls made under different oracle-budget constraints.

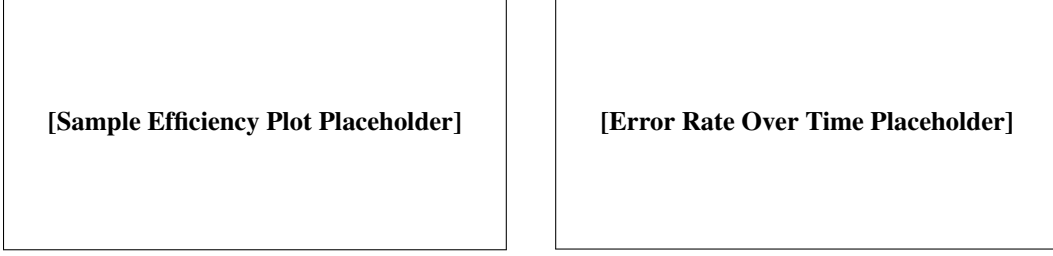


Figure 2: **Left:** Accuracy vs. number of training examples consumed (sample efficiency). **Right:** Error rate over successive SWS cycles. HAT achieves higher accuracy with fewer examples and reduces errors faster than baselines.

Table 2: Impact of oracle budget on HAT performance (Phi-3-3.8B, NQ benchmark). “% Oracle” is the fraction of interactions that trigger an oracle call.

Setting	% Oracle	Accuracy	Error Rate
HAT (no oracle)	0%	—	—
HAT ($\tau_{\mathcal{O}} = 0.9$)	—%	—	—
HAT ($\tau_{\mathcal{O}} = 0.7$)	—%	—	—
HAT ($\tau_{\mathcal{O}} = 0.5$)	—%	—	—
Full Distillation	100%	—	—

298 6 Analysis

299 6.1 Ablation Study

300 To understand the contribution of each component, we ablate key modules of HAT using the Phi-3-
301 3.8B Cortex on the NQ benchmark. Results are summarised in Table 3.

302 **Abstraction is critical.**

303 **Selection outperforms random curation.**

304 **All three selection signals contribute.**

305 **EWC prevents forgetting.**

306 6.2 Effect of SWS Cycle Frequency

307 We vary the number of interactions between SWS cycles ($N \in \{100, 250, 500, 1000, 2000\}$) and
308 measure final accuracy and computational cost.

309 6.3 Qualitative Analysis

310 **What does the Hippocampus select?** We analyse the distribution of memory traces written to the
311 Neocortex across interaction types.

312 **Memory evolution over time.** We visualise the Neocortex content across SWS cycles using t-SNE
313 embeddings.

314 **Case study: error correction propagation.** We trace a specific user correction through the HAT
315 pipeline—from raw interaction to abstracted trace, selection, replay construction, and its effect on
316 model behaviour after SWS training.

317 6.4 Scaling Behaviour

318 We examine how HAT’s gains vary with Cortex model size (1.5B, 3.8B, 8B).

Table 3: Ablation study on the NQ benchmark (Phi-3-3.8B). Each row removes or replaces one component. Δ indicates the accuracy change relative to the full HAT system.

Variant	Accuracy	Δ
HAT (full)	—	—
w/o Abstraction	—	—
w/o Selection (random write)	—	—
w/o Novelty signal ($\gamma = 0$)	—	—
w/o Uncertainty signal ($\alpha = 0$)	—	—
w/o Feedback signal ($\beta = 0$)	—	—
w/o Oracle consultation	—	—
w/o EWC regularisation	—	—
w/o Replay Builder (raw traces)	—	—

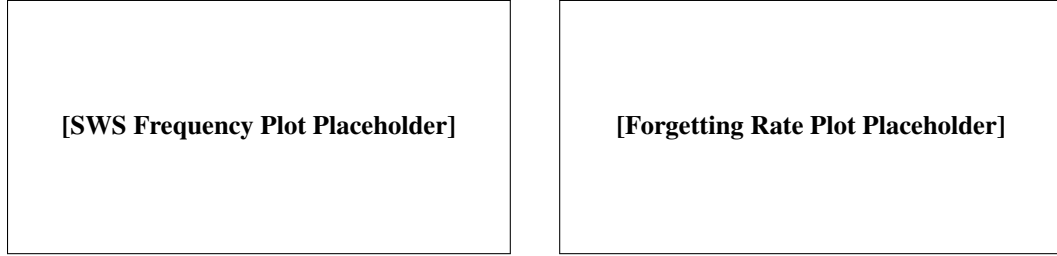


Figure 3: **Left:** Final accuracy as a function of SWS cycle frequency. **Right:** Forgetting rate across SWS cycles for HAT vs. baselines.

7 Conclusion

We introduced **HAT (Hippocampus-Augmented Transformer)**, a memory-centric framework that enables lightweight language models to self-improve by learning *what to learn* from interactive experience. Drawing on complementary learning systems theory, HAT decouples experience acquisition (online) from parameter optimisation (offline) through three key mechanisms: abstraction of uncertain interactions into structured memory traces, embedding-based CREATE/REVISE routing against a curated Neocortex, and offline replay-driven fine-tuning during slow-wave sleep phases. An on-demand oracle consultation mechanism provides efficient, targeted knowledge distillation only where the model is most uncertain.

Experiments across knowledge-intensive QA, instruction following, and personalisation benchmarks demonstrate that HAT consistently outperforms strong fine-tuning and retrieval-augmented baselines in accuracy, sample efficiency, and resistance to catastrophic forgetting, while using a fraction of the oracle budget required by full distillation.

Limitations and future work. Several directions remain open. First, the current write policy uses a fixed uncertainty threshold (Eq. 17) and a fixed semantic routing threshold (Eq. 16); learning these policies end-to-end via meta-learning is a promising extension. Second, our evaluation simulates user interactions from existing benchmarks; deploying HAT with real users in a longitudinal study would provide stronger evidence of its practical impact. Third, while HAT targets lightweight models (0.5–10 B), scaling the framework to larger models and investigating the interplay between model capacity and memory curation is of both theoretical and practical interest. Finally, privacy considerations arise when storing user interactions in the Neocortex; integrating differential privacy or federated learning into the SWS training phase is an important direction for responsible deployment.

References

Elahe Arani, Fahad Sarfraz, and Bahram Zonooz. Learning fast, learning slow: A general continual learning method based on complementary learning system. In *International Conference on Learning Representations*, 2022.

345 Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones,
346 Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. Constitutional AI:
347 Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022.

348 Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican,
349 George Bm Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al.
350 Improving language models by retrieving from trillions of tokens. *arXiv preprint arXiv:2112.04426*,
351 2022.

352 Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal,
353 Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are
354 few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages
355 1877–1901, 2020.

356 Pietro Buzzega, Matteo Boschini, Angelo Porrello, Davide Abati, and Simone Calderara. Dark
357 experience for general continual learning: A strong, simple baseline. In *Advances in Neural
358 Information Processing Systems*, volume 33, pages 15920–15930, 2020.

359 Zixiang Chen, Yihe Deng, Huizhuo Yuan, Kaixuan Ji, and Quanquan Gu. Self-play fine-tuning
360 converts weak language models to strong language models. *arXiv preprint arXiv:2401.01335*,
361 2024.

362 Matthias De Lange, Rahaf Aljundi, Marc Masana, Sarah Parisot, Xu Jia, Aleš Leonardis, Gregory
363 Slabaugh, and Tinne Tuytelaars. A continual learning survey: Defying forgetting in classification
364 tasks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(7):3366–3385, 2021.

365 Alex Graves, Greg Wayne, and Ivo Danihelka. Neural turing machines. In *arXiv preprint
366 arXiv:1410.5401*, 2014.

367 Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Mingwei Chang. REALM: Retrieval-
368 augmented language model pre-training. In *International Conference on Machine Learning*, pages
369 3929–3938, 2020.

370 Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv
371 preprint arXiv:1503.02531*, 2015.

372 Jiaxin Huang, Shixiang Shane Gu, Le Hou, Yuexin Wu, Xuezhi Wang, Hongkun Yu, and Jiawei Han.
373 Large language models can self-improve. *arXiv preprint arXiv:2210.11610*, 2023.

374 Gautier Izacard, Patrick Lewis, Maria Lomeli, Lucas Hosseini, Fabio Petroni, Timo Schick, Jane
375 Dwivedi-Yu, Armand Joulin, Sebastian Riedel, and Edouard Grave. Atlas: Few-shot learning
376 with retrieval augmented language models. *Journal of Machine Learning Research*, 24(251):1–43,
377 2023.

378 Mandar Joshi, Eunsol Choi, Daniel S Weld, and Luke Zettlemoyer. TriviaQA: A large scale distantly
379 supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual
380 Meeting of the Association for Computational Linguistics*, pages 1601–1611, 2017.

381 Zixuan Ke, Yijia Shao, Haowei Lin, Tatsuya Konishi, Gyuhak Kim, and Bing Liu. Continual
382 pre-training of language models. *arXiv preprint arXiv:2302.03241*, 2023.

383 James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A
384 Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. Overcoming
385 catastrophic forgetting in neural networks. In *Proceedings of the National Academy of Sciences*,
386 volume 114, pages 3521–3526, 2017.

387 Dharshan Kumaran, Demis Hassabis, and James L McClelland. What learning systems do intelligent
388 agents need? Complementary learning systems theory updated. *Trends in Cognitive Sciences*, 20
389 (7):512–534, 2016.

390 Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris
391 Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. Natural questions: A
392 benchmark for question answering research. *Transactions of the Association for Computational
393 Linguistics*, 7:453–466, 2019.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.

Xuechen Li, Tianyi Zhang, Yann Dubois, Rohan Taori, Ishaan Gulrajani, Carlos Guestrin, Percy Liang, and Tatsunori B Hashimoto. AlpacaEval: An automatic evaluator of instruction-following models. *GitHub repository*, 2023.

Zhizhong Li and Derek Hoiem. Learning without forgetting. In *IEEE Transactions on Pattern Analysis and Machine Intelligence*, volume 40, pages 2935–2947, 2017.

James L McClelland, Bruce L McNaughton, and Randall C O’Reilly. Why there are complementary learning systems in the hippocampus and neocortex: Insights from the successes and failures of connectionist models of learning and memory. *Psychological Review*, 102(3):419–457, 1995.

OpenAI. GPT-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744, 2022.

Björn Rasch and Jan Born. About sleep’s role in memory. *Physiological Reviews*, 93(2):681–766, 2013.

Alireza Salemi, Sheshera Mysore, Michael Bendersky, and Hamed Zamani. LaMP: When large language models meet personalization. *arXiv preprint arXiv:2304.11406*, 2024.

Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter. In *NeurIPS EMC² Workshop*, 2019.

Thomas Scialom, Tuhin Chakrabarty, and Smaranda Muresan. Fine-tuned language models are continual learners. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 6fenêtre–6102, 2022.

Hanul Shin, Jung Kwon Lee, Jaehong Kim, and Jiwon Kim. Continual learning with deep generative replay. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Sainbayar Sukhbaatar, Arthur Szlam, Jason Weston, and Rob Fergus. End-to-end memory networks. In *Advances in Neural Information Processing Systems*, volume 28, 2015.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

Gido M van de Ven, Hava T Siegelmann, and Andreas S Tolias. Brain-inspired replay for continual learning with artificial neural networks. *Nature Communications*, 11(1):4069, 2020.

Matthew Walker. *Why We Sleep: Unlocking the Power of Sleep and Dreams*. Scribner, 2017.

Tongtong Wu, Linhao Luo, Yuan-Fang Li, Shirui Pan, Thuy-Trang Vu, and Gholamreza Haffari. Continual learning for large language models: A survey. *arXiv preprint arXiv:2402.01364*, 2024.

Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. Self-rewarding language models. *arXiv preprint arXiv:2401.10020*, 2024.

Friedemann Zenke, Ben Poole, and Surya Ganguli. Continual learning through synaptic intelligence. In *International Conference on Machine Learning*, pages 3987–3995, 2017.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging LLM-as-a-judge with MT-Bench and chatbot arena. *Advances in Neural Information Processing Systems*, 36, 2023.

Zexuan Zhong, Tao Lei, and Danqi Chen. Training language models with memory augmentation. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 5657–5673, 2022.

442 **A Algorithm Pseudocode**

Algorithm 2: HAT — Hippocampus-Augmented Transformer Self-Improvement Loop

Input : Base model $\mathcal{M}_\theta$ (Cortex), Hippocampus Agent $\mathcal{H}$, Oracle $\mathcal{O}$, empty Neocortex $\mathcal{N} \leftarrow \emptyset$
Input : SWS interval N , write threshold τ , oracle threshold $\tau_{\mathcal{O}}$, selection weights α, β, γ

```

1  $t \leftarrow 0$ 
2 for each user interaction  $(c, x)$  do
3    $y \leftarrow \mathcal{M}_\theta(c, x)$  // Cortex generates response
4    $U \leftarrow \mathcal{H}(p_\theta(y | c, x))$  // Estimate uncertainty
5   Observe user feedback  $F$  (correction signal, if any)
6   // -- Hippocampus Agent --
7    $m \leftarrow \mathcal{H}_{\text{abs}}(c, x, y)$  // Abstraction
8   if  $U > \tau_{\mathcal{O}}$  or  $F = 1$  then
9      $y^{\mathcal{O}} \leftarrow \mathcal{O}(c, x)$  // Consult Oracle
10    Update  $m$  with oracle response  $y^{\mathcal{O}}$ 
11    Compute novelty:  $N \leftarrow \text{NoveltyScore}(m, \mathcal{N})$ 
12    Compute selection score:  $s \leftarrow \alpha U + \beta F + \gamma N$ 
13    if  $s > \tau$  then
14       $(x^{\text{train}}, y^{\text{train}}) \leftarrow \mathcal{H}_{\text{replay}}(m)$  // Replay Builder
15       $\mathcal{N} \leftarrow \mathcal{N} \cup \{(x^{\text{train}}, y^{\text{train}}, s)\}$  // Write to Neocortex
16     $t \leftarrow t + 1$ 
17    // -- Slow-Wave Sleep --
18    if  $t \bmod N = 0$  then
19      Sample replay batch  $\mathcal{B} \sim \mathcal{N}$  (priority sampling by  $s$ )
20       $\theta \leftarrow \theta - \eta \nabla_\theta [\mathcal{L}_{\text{SWS}}(\theta; \mathcal{B}) + \mathcal{L}_{\text{reg}}(\theta)]$  // SWS update
21      Update Fisher information  $F$  for EWC
22 return Updated model  $\mathcal{M}_\theta$ 

```

443 **B Hyperparameter Sensitivity**

Table 4: Hyperparameter sensitivity analysis on the NQ benchmark (Phi-3-3.8B).

Hyperparameter	Range Tested	Best Value	Sensitivity
Write threshold τ	{0.1, 0.2, 0.3, 0.5, 0.7}	0.3	Medium
Oracle threshold $\tau_{\mathcal{O}}$	{0.5, 0.6, 0.7, 0.8, 0.9}	0.7	Low
Uncertainty weight α	{0.2, 0.3, 0.4, 0.5, 0.6}	0.4	Medium
Feedback weight β	{0.2, 0.3, 0.4, 0.5, 0.6}	0.4	Low
Novelty weight γ	{0.1, 0.2, 0.3, 0.4}	0.2	Low
LoRA rank	{4, 8, 16, 32}	16	Low
SWS learning rate η	$\{1, 2, 5\} \times 10^{-5}$	2×10^{-5}	Medium
EWC coefficient μ	{0.01, 0.05, 0.1, 0.5, 1.0}	0.1	Medium
SWS interval N	{100, 250, 500, 1000, 2000}	500	Low

Table 5: Computational overhead of HAT components (Phi-3-3.8B on a single NVIDIA A100 GPU).

Component	Time per Interaction	Time per SWS Cycle
Cortex inference	– ms	—
Hippocampus (abstraction)	– ms	—
Hippocampus (selection)	– ms	—
Oracle consultation	– ms	—
SWS training (3 epochs)	—	– min
Total overhead	– ms	– min

444 C Additional Experimental Results

445 D Prompts Used by the Hippocampus Agent

446 E Computational Cost Analysis

447 F Broader Impact Statement

448 HAT enables on-device language models to improve from user interactions, which raises considera-
 449 tions in both positive and negative directions.

450 **Positive impacts.** HAT democratises model adaptation by allowing lightweight models on personal
 451 devices to improve without requiring cloud-based retraining. This can reduce reliance on centralised
 452 AI services and give users more control over their AI assistants. The selective memory mechanism
 453 also implies that not all data is stored, which can be beneficial for privacy.

454 **Potential risks.** Storing user interactions, even in abstracted form, poses privacy risks. If the
 455 Neocortex memory is compromised, sensitive information could be exposed. Additionally, a model
 456 that learns from user corrections could amplify biases if the corrections themselves are biased. We
 457 recommend deploying HAT with differential privacy mechanisms and content filtering to mitigate
 458 these risks.

NeurIPS Paper Checklist

The checklist is designed to encourage best practices for responsible machine learning research, addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove the checklist: **The papers not including the checklist will be desk rejected.** The checklist should follow the references and follow the (optional) supplemental material. The checklist does NOT count towards the page limit.

Please read the checklist guidelines carefully for information on how to answer these questions. For each question in the checklist:

- You should answer [Yes], [No], or [N/A].
- [N/A] means either that the question is Not Applicable for that particular paper or the relevant information is Not Available.
- Please provide a short (1–2 sentence) justification right after your answer (even for [N/A]).

The checklist answers are an integral part of your paper submission. They are visible to the reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it (after eventual revisions) with the final version of your paper, and its final version will be published with the paper.

The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation. While [Yes] is generally preferable to [No], it is perfectly acceptable to answer [No] provided a proper justification is given (e.g., error bars are not reported because it would be too computationally expensive” or “we were unable to find the license for the dataset we used”). In general, answering [No] or [N/A] is not grounds for rejection. While the questions are phrased in a binary way, we acknowledge that the true answer is often more nuanced, so please just use your best judgment and write a justification to elaborate. All supporting evidence can appear either in the main paper or the supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification please point to the section(s) where related material for the question can be found.

IMPORTANT, please:

- Delete this instruction block, but keep the section heading “NeurIPS Paper Checklist”.
- Keep the checklist subsection headings, questions/answers and guidelines below.
- Do not modify the questions and only use the provided macros for your answers.

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper does not include experiments.

- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).

- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer **[N/A]** means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer **[N/A]** means that the paper does not include experiments.
- The authors should answer **[Yes]** if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer **[N/A]** means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.

- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper poses no such risks.

- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.